🛠️ Ejercicios

Tabla de contenidos

1 El estante de la Farmacia 2
1.1 Contexto 2
1.2 Reglas de costo 2
1.3 Suposiciones clave 2
1.4 Tareas de análisis y cálculo 3
2 Cita con el dentista 3
2.1 Contexto 3
2.2 Datos iniciales 3
2.3 Funciones a implementar 4
2.4 Ejemplo de uso 4
2.5 Casos a probar 4
2.6 Mini-consigna teórica 5
3 La lista de reproducción del DJ 5
3.1 Contexto 5
3.2 Requerimientos de manipulación 5
3.3 Datos iniciales 5
3.4 Ejemplos de uso 5
3.5 Punto extra: Visualización del Índice 6
4 Scanner Pro 3000 6
4.1 Contexto 6
5 El club secreto de los *hackers* 7
5.1 Contexto 7
5.2 Datos iniciales 7
5.3 Ejemplo de uso esperado 8
5.4 Ejemplo de uso esperado 8
5.5 Nivel 2: Desafío 🚀 8
5.6 Ejemplo de uso 9
6 Torre de control aéreo 9
6.1 Contexto 9
6.2 Prioridad de la torre 9
6.3 Datos iniciales 10
6.4 Requisitos funcionales 10
7 Alerta de virus zombie 10
7.1 Contexto 10
7.2 Datos iniciales 10
7.3 Tareas 10
7.4 Ejemplo de uso esperado 11

7.5 Casos a probar 11
7.6 Nivel 2: Desafío 🚀 12
7.7 Nueva funcionalidad: Muestras prioritarias 12
7.8 Datos iniciales 12
7.9 Tareas adicionales 12
7.10 Ejemplo de uso 12
8 Batalla Pokémon 13
8.1 Contexto 13
8.2 Ejemplo de uso 14
8.3 Requerimientos para implementar la clase `Equipo` 14
8.4 Métodos necesarios 14
8.5 Ejemplo de uso completo 15
9 Uno más dos 16
10 Uno más tres 17
11 Más y más datos 17
12 El regreso de Fibonacci 17
13 Vinieron los primos 18
14 El bueno, el malo y el eficiente 18
15 Seamos sinceros 18
16 Seamos originales 19
17 Altas dimensiones 19
18 Mediana complejidad 19
19 *Bubble sort* eficiente 20
20 Piedra *sort* 20
21 *Selection sort* basado en máximos 20

1 El estante de la Farmacia

1.1 Contexto

Imaginá un estante de farmacia con N posiciones perfectamente contiguas (sin huecos), numeradas desde 0 hasta $N - 1$. Cada posición guarda exactamente una caja de un medicamento.

Para este ejercicio, asumiremos que N = 100.

1.2 Reglas de costo

Para medir el costo de cada operación, definimos un paso como la unidad de tiempo más pequeña:

1. **Observar (Leer):** Mirar qué medicamento hay en una posición específica. **(1 paso)**
2. **Mover:** Deslizar una caja una posición a la izquierda o derecha. **(1 paso por cada caja movida)**
3. **Colocar (Escribir):** Poner una caja nueva en una posición vacía. **(1 paso)**

1.3 Suposiciones clave

- **Inserción:** Para hacer un hueco en medio del estante, hay que mover todas las cajas desde esa posición en adelante hacia la derecha.

- **Eliminación:** Para cerrar un hueco en medio del estante, hay que mover todas las cajas desde la posición eliminada en adelante hacia la izquierda.
- El estante debe permanecer siempre contiguo (sin huecos).

1.4 Tareas de análisis y cálculo

Para cada escenario, calcule el costo en pasos y deduzca su complejidad asintótica (*Big* O), basándote en la analogía física.

Operación	Descripción del escenario	Pasos (Con N=100)	Expresión en N (Big O implícito)	Justificación (¿Por qué?)
a) Lectura por Posición	Decime qué medicamento hay en la posición **37**.			
b) Búsqueda (peor caso)	¿Está el medicamento X en el estante? (Asumí que **NO está** y debes revisar todo el estante).			
c) Inserción al Inicio	Poné un nuevo medicamento en la **posición 0** y corré todo lo demás.			
d) Inserción al Final	Agregá un nuevo medicamento en la **posición N** (al final del estante).			
e) Eliminación al Inicio	Sacá la caja de la **posición 0** y corré el resto para cerrar el hueco.			
f) Eliminación al Final	Sacá la última caja del estante (posición N−1).			

2 Cita con el dentista

2.1 Contexto

Una clínica odontológica necesita un sistema sencillo para gestionar su agenda de turnos. Los turnos se agregan a la lista en orden de llegada, sin preocuparse inicialmente por el orden cronológico.

2.2 Datos iniciales

La agenda se mantiene en un arreglo (lista) de diccionarios.

```
agenda = [
    {"paciente": "Juana Pérez", "hora": "08:30"},
    {"paciente": "Mario Gómez", "hora": "09:00"},
    {"paciente": "Sofía López", "hora": "09:30"},
    {"paciente": "Ana Fernández", "hora": "10:15"},
]
```

2.3 Funciones a implementar

Usando operaciones básicas sobre el arreglo (sin usar `.remove` ni `.pop` directamente):

- `leer_pos(i)`: Devuelve el turno en la posición `i`. Si `i` no existe, devuelve `None`.
- `insertar(turno)`: Agrega un nuevo diccionario `{"paciente": ..., "hora": "HH:MM"}` al final de la lista (simulando la llegada de un nuevo paciente). Devuelve `True`.
- `eliminar(hora)`: Busca el primer turno con esa hora y lo elimina. Devuelve `True` si lo eliminó, `False` si el horario no estaba en la lista.
- `buscar(hora)`: Recorre la lista desde el principio hasta el final. Devuelve la posición (índice) donde está ese horario, o `-1` si no existe.
- `listar()`: Devuelve una lista de strings con el formato: `["0) 08:30 - Juana Pérez", "1) 09:00 - Mario Gómez", ...]`

2.4 Ejemplo de uso

```
print(leer_pos(1))
# {'paciente': 'Mario Gómez', 'hora': '09:00'}

nuevo = {"paciente": "Diego Romero", "hora": "09:45"}
insertar(nuevo)  # Se agrega al final

print(buscar("09:45"))
# 4  (Si era el quinto elemento, ya que la lista creció)

print(eliminar("09:00"))
# True

print(listar())
# ['0) 08:30 - Juana Pérez',
#  '1) 09:30 - Sofía López',
#  '2) 10:15 - Ana Fernández',
#  '3) 09:45 - Diego Romero']
```

2.5 Casos a probar

- Insertar `"08:15"` (debe ir al inicio).
- Insertar `"12:00"` (debe ir al final).
- Insertar repetido `"09:30"` (debe rechazarse).
- Buscar un horario inexistente (debe devolver `-1`).
- Eliminar uno inexistente (debe devolver `False`).

2.6 Mini-consigna teórica

Responda:

- ¿Qué ocurre con los índices cuando eliminás o insertás un turno en el medio de la lista? (Pista: ¿qué pasa con los elementos que estaban después?)
- ¿Por qué acceder a `agenda[i]` es una operación $O(1)$ pero buscar un turno por hora es $O(N)$?
- ¿Qué estructura te parecería más conveniente si hubiera miles de turnos? Explicá brevemente por qué.

3 La lista de reproducción del DJ

3.1 Contexto

Un DJ está gestionando su lista principal de canciones, almacenada en una secuencia ordenada de reproducción. Se necesita desarrollar las herramientas para manipular esta lista de forma eficiente y precisa antes del show.

3.2 Requerimientos de manipulación

El sistema debe soportar las siguientes acciones, considerando que la lista es una secuencia lineal donde el orden es fundamental:

- **Consulta por ubicación**: El DJ debe poder pedir la canción que está en una posición exacta de la secuencia (ej. "Dime qué canción está en el tercer lugar").
- **Adición controlada**: Se debe poder insertar una nueva canción exactamente en una posición específica de la secuencia (ej. "Pon 'Nueva Canción' justo antes de la canción que actualmente está en la posición 5"). Esto implica que las canciones que estaban en esa posición y después deben moverse para hacer espacio.
- **Remoción dirigida**: El DJ debe poder quitar una canción conociendo su nombre de la lista. Una vez retirada, las canciones que estaban después deben moverse para cerrar el hueco y mantener la secuencia contigua.
- **Identificación por nombre**: Se requiere una función para localizar una canción por su nombre y devolver su ubicación actual en la secuencia.

3.3 Datos iniciales

```
canciones = ["Thunderstruck", "Billie Jean", "Smells Like Teen Spirit"]
```

3.4 Ejemplos de uso

```
# Ejemplo de requerimiento: Inserción
# Intentamos poner "One" en la posición 2 (el tercer espacio)
insertar_cancion(2, "One")
# La lista debe ajustarse automáticamente: ["Thunderstruck", "Billie Jean",
"One", "Smells Like Teen Spirit"]

# Ejemplo de requerimiento: Eliminación
```

```
eliminar_cancion("Billie Jean")
# La lista debe reajustarse: ["Thunderstruck", "One", "Smells Like Teen
Spirit"]
```

3.5 Punto extra: Visualización del Índice

Muestre al DJ la lista completa, indicando claramente la posición inicial de cada pista en la secuencia.

4 Scanner Pro 3000

4.1 Contexto

En un recital, cada entrada tiene un código. Al ingresar, el lector va guardando los códigos en una lista en el orden en que se escanean. La producción te pide detectar si hubo códigos repetidos (fraude o doble escaneo).

Te entregan una lista escaneos con miles de códigos (que son *strings* de Python).

```
escaneos = [
    "A1F3", "B9K2", "X7Z1", "A1F3", "L0P9", "M2Q5", "B9K2", ...
]
```

1. Escriba una función que encuentre si hay algún código repetido comparando cada elemento con todos los que le siguen (utilice un doble bucle `for`).
   - ¿Cuántas comparaciones hace en el peor caso con N elementos?
   - ¿Qué pasa cuando N crece x10?
2. Implemente una versión lineal usando un `set` para registrar códigos ya vistos.
   - ¿Por qué ahora es $O(N)$?
   - ¿Qué operación te permite "saltar" comparaciones?
3. Use esta funcion que crea codigos aleatorios para generar listas de codigos de tamaños crecientes y mida el tiempo de ejecucion de cada funcion anterior: ```python import random, string, time

   def generar_codigos(n, dup_ratio=0.0, seed=0): rng = random.Random(seed) base = set() # crear códigos únicos while len(base) < int(n * (1 - dup_ratio)): base.add(''.join(rng.choices(string.ascii_uppercase + string.digits, k=6))) lista = list(base) # inyectar duplicados while len(lista) < n: lista.append(rng.choice(list(base))) rng.shuffle(lista) return lista ```

5 El club secreto de los *hackers*

5.1 Contexto

Un grupo de *hackers* tiene un club secreto con reglas muy estrictas sobre quién puede entrar y salir. Los miembros se ordenan según su entrada: desde el fundador hasta el último ingresante. En caso de ser necesaria una reducción de miembros, los primeros en irse deben ser los que menos experiencia tienen (el último en entrar al club debe ser el primero en salir).

Si un hacker que no es el último en entrar sale del grupo fuera de orden, se considera una violación de seguridad y se registra la acción en el sistema.

5.2 Datos iniciales

```
club = [
    {"nombre": "Anonymous"},
    {"nombre": "Mitnick"},
    {"nombre": "Soupnazi"},
    {"nombre": "D-Dante"}
]
```

Teniendo en cuenta el contexto implementa las siguientes funciones para regular la entrada y salida de los miembros del club:

1. `entrar(nombre)`
   - Agrega un hacker a la pila del club
   - Cada entrada debe registrarse como un diccionario: `{"nombre": "Neo"}`
   - Si hay más de 10 hacker, imprimir: **“El Club esta lleno, capacidad máxima alcanzada”**
2. `salir()`
   - Saca al último hacker que entró
   - Retorna el nombre del hacker que salió
   - Si la pila está vacía, imprimir: **“El club ha desaparecido en la oscuridad…”**
3. `ultimo_en_entrar()`
   - Muestra quién está en la cima de la pila **sin sacarlo**
   - Retorna el nombre del último que entró
   - Si la posiion aun no fue cubierta, retorna `None`
4. `mostrar_club()`
   - Muestra todos los hackers en el club, del más reciente al más antiguo
   - Indica cuántos hay en total
   - Formato: `[Top] Neo -> Trinity -> Anonymous [Base]`
5. `esta_en_club(nombre)`
   - Verifica si un hacker específico está dentro
   - Retorna `True` o `False`
   - **IMPORTANTE**: No destruir la pila al buscar

5.3 Ejemplo de uso esperado

```
entrar("Morpheus")
entrar("Trinity")
entrar("Neo")

mostrar_club()
# Salida:  Club (3 hackers): [Top] Neo -> Trinity -> Morpheus [Base]

print(ultimo_en_entrar())
# Salida: Neo

salir()
# Salida: Neo ha salido del club

print(esta_en_club("Trinity"))
# Salida: True

print(esta_en_club("Cypher"))
# Salida: False
```

5.4 Ejemplo de uso esperado

```
# Caso 1: Club vacío
club.clear()
salir()  # Debe mostrar mensaje

# Caso 2: Verificar LIFO
entrar("A")
entrar("B")
entrar("C")
salir()  # Debe salir C (último en entrar)
salir()  # Debe salir B
ultimo_en_entrar()  # Debe ser A

# Caso 3: Club lleno
for i in range(12):
    entrar(f"Hacker{i}")  # En el 11vo debe avisar que está lleno

# Caso 4: Buscar sin destruir
entrar("Neo")
entrar("Trinity")
print(esta_en_club("Neo"))  # True
mostrar_club()  # Neo debe seguir en la pila
```

5.5 Nivel 2: Desafío 🚀

Nueva funcionalidad: Control de acceso con violaciones

Impelmente las siguentes funciones:

6. `salir_especifico(nombre)`

- Si nombre es el TOP: sacarlo sin registrar violación, retornar `True`.
- Si no es el TOP pero está en la pila: - Desapilar a un buffer hasta encontrar nombre. - Remover nombre. - Reapilar en orden original todo lo que sacaste.
- Registrar violación en `_log_violaciones` con formato: `"{HH:MM} - Violación: {nombre} salió fuera de orden"` (usar `datetime.now().strftime("%H:%M")`). Retornar `True`.
- Si no se encuentra: retornar `False`.

7. `historial_violaciones()`
   - Muestra todas las violaciones de seguridad registradas
   - Formato: `["17:30 - Violación: Trinity salió fuera de orden"]`

5.6 Ejemplo de uso

```
entrar("Morpheus")
entrar("Trinity")
entrar("Neo")
entrar("Cypher")

# Neo y Cypher están encima de Trinity
salir_especifico("Trinity")
# Salida: VIOLACIÓN DE SEGURIDAD: Trinity salió fuera de orden

mostrar_club()
# Salida: [Top] Cypher -> Neo -> Morpheus [Base]
# Trinity ya no está
```

6 Torre de control aéreo

6.1 Contexto

La Autoridad de Tráfico Aéreo necesita optimizar las operaciones de una torre de control. Tu tarea es modelar el sistema que gestiona los permisos de movimiento de los aviones.

Existen dos tipos de solicitudes pendientes:

- **Aviones esperando despegue:** Estos aviones forman una fila de espera. Se les debe dar permiso para despegar siguiendo un principio estricto: el avión que lleva más tiempo esperando en la fila es el primero en recibir permiso.
- **Aviones en aproximación de emergencia:** Estos aviones requieren aterrizaje inmediato debido a una situación crítica. Estos permisos de aterrizaje se apilan, y la torre debe procesar la emergencia más reciente antes que cualquier otra, es decir: el último avión en reportar la emergencia es el primero en recibir la autorización de aterrizaje.

6.2 Prioridad de la torre

La torre de control siempre otorga permisos siguiendo una estricta jerarquía:

- **Prioridad absoluta:** Si hay alguna emergencia pendiente, la torre siempre debe autorizar primero un aterrizaje de emergencia.
- **Prioridad secundaria:** Solo cuando no haya ninguna emergencia pendiente, la torre podrá autorizar el despegue del avión que le corresponda.

6.3 Datos iniciales

Comienza con la siguiente situación en el aeropuerto:

```
# Lista de aviones esperando para Despegar
solicitudes_despegue = ["Vuelo 101", "Vuelo 205"]
# Lista de aviones reportando Emergencia
solicitudes_emergencia = ["Vuelo 999"]
```

6.4 Requisitos funcionales

Se debe crear un sistema con las siguientes acciones:

- Una función para registrar la llegada de un nuevo avión a la lista de despegue.
- Una función para registrar una nueva solicitud de aterrizaje de emergencia.
- Una función principal, `procesar_evento()`, que simule la acción de la torre de control al otorgar un permiso según las reglas de prioridad.

Punto extra: Si la torre intenta otorgar un permiso y ambas listas están vacías, debe notificarlo con el mensaje: `"Torre en silencio"`.

7 Alerta de virus zombie

7.1 Contexto

Sos un científico/a de un laboratorio secreto que está procesando muestras de virus zombies. Debes implementar un sistema de gestión usando colas para manejar el flujo de trabajo.

Alerta importante: Si hay más de 5 muestras del mismo tipo de virus en la cola, se activa una alerta biológica (riesgo de propagación masiva).

7.2 Datos iniciales

```
from collections import deque

muestras = deque(["virus_tank", "virus_witch", "virus_charger"])
```

7.3 Tareas

Implementa las siguientes funciones:

1. `agregar_muestra(nombre)`
   - Agrega una muestra al final de la cola
   - Muestra mensaje confirmando la operación
   - Verifica si hay más de 5 muestras del mismo tipo

- Si se cumple la condición, imprime: `"¡Alerta biológica! Detectadas X muestras de [nombre_virus]"`

2. `procesar_muestra()`
   - Procesa (elimina) la primera muestra de la cola
   - Retorna el nombre de la muestra procesada
   - Si la cola está vacía, muestra un mensaje de error
3. `mostrar_pendientes()`
   - Muestra la lista de muestras pendientes
   - Retorna la lista de muestras
   - Indica cuántas muestras hay en espera
4. `contar_por_tipo()`
   - Retorna un diccionario con el conteo de cada tipo de virus
   - Ejemplo: `{"virus_tank": 3, "virus_witch": 2}`

7.4 Ejemplo de uso esperado

```
print(mostrar_pendientes())
# Salida: Muestras pendientes (3): ['virus_tank', 'virus_witch',
'virus_charger']

agregar_muestra("virus_tank")
agregar_muestra("virus_tank")
agregar_muestra("virus_tank")
agregar_muestra("virus_tank")
agregar_muestra("virus_tank")
# Salida en la 5ta: ¡Alerta biológica! Detectadas X muestras de virus_tank ☣

print(contar_por_tipo())
# Salida: {'virus_tank': 6, 'virus_witch': 1, 'virus_charger': 1}

procesar_muestra()
# Salida: Procesando 'virus_tank'
# Retorna: 'virus_tank'
```

7.5 Casos a probar

```
# Caso 1: Cola vacía
muestras.clear()
procesar_muestra()  # Debe mostrar error

# Caso 2: Alerta biológica por acumulación
for i in range(6):
    agregar_muestra("virus_hunter")  # Debe activar alerta en la 6ta

# Caso 3: Verificar que la alerta sea por tipo específico
agregar_muestra("virus_tank")
agregar_muestra("virus_witch")
```

```
agregar_muestra("virus_tank")
# No debe haber alerta (solo 2 de cada tipo)

# Caso 4: Procesar reduce el conteo
for i in range(7):
    agregar_muestra("virus_smoker")  # Alerta: 7 smoker
procesar_muestra()  # Procesa 1 virus_tank (de los iniciales)
print(contar_por_tipo())  # Debería seguir mostrando 7 smoker
```

7.6 Nivel 2: Desafío 🚀

7.7 Nueva funcionalidad: Muestras prioritarias

Algunas muestras son urgentes y deben procesarse antes que las normales.

7.8 Datos iniciales

```
from collections import deque

muestras_normales = deque(["virus_tank", "virus_witch"])
muestras_urgentes = deque()
```

7.9 Tareas adicionales

5. `agregar_muestra(nombre, urgente=False)`
   - Si `urgente=True`, agrega a la cola de urgentes
   - Si `urgente=False`, agrega a la cola normal
   - **La alerta biológica debe contar en AMBAS colas** (misma cepa puede estar en las dos)
6. `procesar_muestra()` (modificada)
   - Primero procesa muestras urgentes
   - Si no hay urgentes, procesa normales
   - Si ambas colas están vacías, muestra error
7. `mostrar_todas()`
   - Muestra ambas colas por separado
   - Indica cuántas muestras hay en cada una
   - Muestra el conteo total por tipo de virus
8. `contar_por_tipo()` (modificada)
   - Debe contar en ambas colas
   - Retorna el conteo global de cada tipo

7.10 Ejemplo de uso

```
# Agregar 4 tank normales y 3 tank urgentes
for i in range(4):
    agregar_muestra("virus_tank", urgente=False)
```

```python
for i in range(3):
    agregar_muestra("virus_tank", urgente=True)
    # En la 3ra debe activarse la alerta (total: 4+3=7 tanks)

mostrar_todas()
# Salida:
# Urgentes (3): ['virus_tank', 'virus_tank', 'virus_tank']
# Normales (6): ['virus_tank', 'virus_witch', 'virus_tank', ...]
# Conteo por tipo: {'virus_tank': 7, 'virus_witch': 1}
```

8 Batalla Pokémon

8.1 Contexto

Un gimnasio Pokémon necesita un sistema para gestionar los batallas entre equipos. Para lograrlo te contratan y te dan la siguiente infromación:

- Los entrenadores Pokémon pueden capturar nuevos Pokémones y agregarlos a su equipo
- Los entrenadores necesitan poder buscar un Pokémon específico por su nombre para usarlo en batalla
- Los entrenadores tienen que poder retirar del equipo a los Pokémon debilitados o que ya no quieren usar
- Los entrenadores deben poder revisar su equipo completo para decidir su estrategia

Tu trabajo sera implementar la clase `Equipo`, para eso puedes usar la siguente clase ya implementada:

```python
class Pokemon:
    """Representa un Pokémon individual"""
    def __init__(self, nombre, tipo, nivel, max_hp, ataque):
        self.nombre = nombre
        self.tipo = tipo
        self.nivel = nivel
        self.max_hp = max_hp
        self.hp = max_hp  # Empieza con vida completa
        self.ataque = ataque

    def recibir_daño(self, puntos):
        """Reduce HP, no puede ser negativo"""
        self.hp = max(0, self.hp - puntos)

    def curar(self, puntos):
        """Aumenta HP, no puede superar el máximo"""
        self.hp = min(self.max_hp, self.hp + puntos)

    def esta_debilitado(self):
        """Retorna True si el Pokémon no puede combatir"""
```

```
        return self.hp == 0

    def subir_nivel(self):
        """Sube de nivel y aumenta stats"""
        self.nivel += 1
        self.max_hp += 5
        self.hp = self.max_hp  # Se cura completamente

    def __str__(self):
        return f"{self.nombre} ({self.tipo.capitalize()}) Nv.{self.nivel} [HP:
{self.hp}/{self.max_hp}]"
```

8.2 Ejemplo de uso

```
pikachu = Pokemon("Pikachu", "eléctrico", nivel=25, max_hp=35, ataque=14)
print(pikachu)  # Pikachu (Eléctrico) Nv.25 [HP: 35/35]

pikachu.recibir_daño(10)
print(pikachu)  # Pikachu (Eléctrico) Nv.25 [HP: 25/35]
```

8.3 Requerimientos para implementar la clase `Equipo`

Los maestros/as Pokémon necesitan gestionar su equipo de forma dinámica. Las operaciones que más realizan son:

1. **Agregar** un Pokémon recién capturado (siempre al final)
2. **Buscar** un Pokémon específico por nombre
3. **Eliminar** un Pokémon específico del equipo
4. **Listar** todos los Pokémon para revisarlos
5. **Eliminar todos los debilitados** después de una batalla
6. **Contar** cuántos Pokémon tiene

Importante

- No se te permite usar listas normales de Python con acceso por índice** (`lista[0]`, `lista[2]`, etc.)
- La clase debe ser eficiente para agregar y eliminar elementos
- Podes crear clases auxiliares si lo necesitas (como Nodo).

8.4 Métodos necesarios

- `__init__(nombre_entrenador)`. Inicializa un equipo vacío para el entrenador.
- `agregar_pokemon(pokemon)`. Agrega un Pokémon al final del equipo.
- `buscar(nombre)`. Busca un Pokémon por nombre. Retorna el objeto Pokemon si lo encuentra, `None` si no existe.
- `eliminar(nombre)`. Elimina un Pokémon específico del equipo. Retorna `True` si lo eliminó, `False` si no existía. **Importante:** este metodo debe manejar correctamente los casos de elimnar un elemento que sea el primero, uno del medio o el ultimo.

- `eliminar_debilitados()`. Elimina todos los Pokémon con HP igual a 0. Retorna una lista con los nombres de los eliminados.
- `contar()`. Retorna cuántos Pokémon hay en el equipo.
- `esta_vacio()`. Retorna `True` si no hay ningún Pokémon en el equipo.
- `listar()`. Retorna una lista de cadenas de texto con todos los Pokémon en orden. Formato: `["1. Pikachu (Eléctrico) Nv.25 [HP: 35/35]", "2. Charmander...", ...]`
- `obtener_primero()`. Retorna el primer Pokémon del equipo sin eliminarlo. `None` si está vacío.
- `obtener_mas_fuerte()`. Retorna el Pokémon con mayor ataque. `None` si está vacío.
- `pokemon_por_tipo(tipo)`. Retorna una lista con todos los Pokémon de un tipo específico. Ejemplo: `pokemon_por_tipo("fuego")` → lista con Charmander, Vulpix, etc.

8.5 Ejemplo de uso completo

```
# Crear Pokémon
pikachu = Pokemon("Pikachu", "eléctrico", nivel=25, max_hp=35, ataque=14)
charmander = Pokemon("Charmander", "fuego", nivel=20, max_hp=39, ataque=12)
squirtle = Pokemon("Squirtle", "agua", nivel=22, max_hp=44, ataque=9)
bulbasaur = Pokemon("Bulbasaur", "planta", nivel=21, max_hp=45, ataque=10)
eevee = Pokemon("Eevee", "normal", nivel=18, max_hp=55, ataque=8)

# Crear equipo
equipo_ash = Equipo("Ash")

# Agregar Pokémon
equipo_ash.agregar_pokemon(pikachu)
equipo_ash.agregar_pokemon(charmander)
equipo_ash.agregar_pokemon(squirtle)
equipo_ash.agregar_pokemon(bulbasaur)

print(f"Pokémon en el equipo: {equipo_ash.contar()}")  # 4

# Listar equipo
print("\n=== EQUIPO DE ASH ===")
for info in equipo_ash.listar():
    print(info)

# Buscar un Pokémon
encontrado = equipo_ash.buscar("Squirtle")
if encontrado:
    print(f"\n Encontrado: {encontrado}")

# Obtener el más fuerte
fuerte = equipo_ash.obtener_mas_fuerte()
print(f"\n  Más fuerte: {fuerte}")

# Simular batalla
print("\n=== SIMULANDO BATALLA ===")
```

```python
charmander.recibir_daño(39)  # Charmander se debilita
squirtle.recibir_daño(20)    # Squirtle pierde HP pero sobrevive

print(f"Charmander debilitado: {charmander.esta_debilitado()}")

# Eliminar debilitados
eliminados = equipo_ash.eliminar_debilitados()
print(f"Pokémon eliminados: {eliminados}")  # ['Charmander']

# Ver equipo actualizado
print(f"\nPokémon restantes: {equipo_ash.contar()}")  # 3
for info in equipo_ash.listar():
    print(info)

# Agregar nuevo Pokémon
equipo_ash.agregar_pokemon(eevee)
print(f"\n Eevee agregado. Total: {equipo_ash.contar()}")

# Buscar por tipo
fuego = equipo_ash.pokemon_por_tipo("fuego")
print(f"\nPokémon de fuego: {[str(p) for p in fuego]}")

# Eliminar un Pokémon específico
equipo_ash.eliminar("Pikachu")
print(f"\n Pikachu liberado. Restantes: {equipo_ash.contar()}")
```

Ayuda

Preguntas para reflexionar ANTES de programar

1. ¿Qué estructura me permite conectar elementos sin índices?
2. ¿Necesito crear una clase auxiliar (como Nodo)?
3. Al eliminar un elemento, ¿qué debo hacer con los enlaces?
   - Pista: Debes "saltar" al elemento eliminado reconectando los que estaban antes y después
4. ¿Cómo recorro todos los elementos sin índices?
   - Pista: Empezas por el primero y seguis los enlaces hasta llegar al final
5. ¿Qué pasa si elimino el primer elemento? ¿Es diferente a eliminar del medio?
   - Pista: El primer elemento es especial, no tiene un elemento anterior

9 Uno más dos

Proponer e implementar dos algoritmos que calculen la suma de los primeros `N` números naturales.

- Uno con complejidad $O(N)$ (lineal).
- Uno con complejidad $O(1)$ (constante).

10 Uno más tres

Proponer e implementar dos algoritmos que calculen la suma de los primeros `N` números naturales **impares**.

- Uno con complejidad $O(N)$ (lineal).
- Uno con complejidad $O(1)$ (constante).

11 Más y más datos

a. Considere el siguiente código que calcula un acumulado de promedios para una lista de tamaño `N`.

```python
def promedios_acumulados(lista):
    """
    Ejemplo:  Entrada -> [1, 2, 3, 5]
              Salida  -> [1, 1.5, 2, 2.75]
    """
    N = len(lista)
    promedios = []
    for i in range(1, N+1):
        promedio = sum(lista[:i]) / i
        promedios.append(promedio)
    return promedios
```

Determine la complejidad de este algoritmo.

b. Siendo $\overline{X}_n$ el promedio de los primeros n números en la lista, vale la siguiente identidad recursiva:

$$\overline{X}_{n+1} = \frac{n \cdot \overline{X}_n + x_{n+1}}{n+1}$$

donde x_{n+1} es el elemento $(n+1)$-ésimo de la lista.

Utilice este resultado para implementar un algoritmo más eficiente.

12 El regreso de Fibonacci

En el Ejercicio 4 de la Unidad 2 se implementó una función recursiva para calcular el n-ésimo número en la sucesión de Fibonacci.

Esta implementación, sin embargo, es muy ineficiente. De hecho, se puede demostrar que su complejidad es $O(2^n)$ (exponencial).

Utilice memoización para mejorar la eficiencia del algoritmo recursivo. Este nuevo programa tendrá eficiencia lineal: $O(n)$.

Ayuda

Al inicio de la función se debe crear un diccionario vacío: `valores = {}`.

Ante cada valor $f(n)$ de la sucesión que toque calcular, se deberá chequear si n está entre las claves existentes del diccionario.

- Si no lo está, se deberá calcular recursivamente y luego almacenar su resultado: `valores[n] = f(n)`.
- Si lo está, se devolverá el valor `f(n)` sin efectuar más cálculos.

13 Vinieron los primos

Implementar un algoritmo para determinar si un número natural `N` es primo, con complejidad $O\left(\sqrt{N}\right)$.

Ayuda

Nótese que, dado un número natural N, para verificar su primalidad es suficiente con verificar si es divisible por algún entero entre 1 y $\sqrt{N}$.

Si fuese un número compuesto, jamás podría expresarse como el producto de dos números mayores a $\sqrt{N}$.

$$a > \sqrt{N}, \quad b > \sqrt{N} \Longrightarrow a \cdot b > \sqrt{N} \cdot \sqrt{N} = N$$

14 El bueno, el malo y el eficiente

Se tiene una lista con `N` números enteros. Se quiere saber si en la lista existe al menos un par de elementos tales que su suma sea impar.

Proponga e implemente dos algoritmos para esta tarea:

- Uno con complejidad $O(N^2)$ (cuadrática).
- Uno con complejidad $O(N)$ (lineal).

15 Seamos sinceros

Utilizando el módulo `time`, compare el tiempo que tardan estos dos métodos para determinar si una lista incluye ceros:

- Un bucle `for` que evalúa cada elemento.
- `0 in mi_lista`

¿Cuál es más rápido? ¿Qué complejidad parece tener cada uno?

Ayuda

Utilice la función `random.randint` de NumPy para generar un arreglo grande con números al azar.

```
import numpy as np

N = 3

# Arreglo de números enteros aleatorios
mi_lista = np.random.randint(0, 10**N, size=10**N)
```

Haga variar el valor de `N` entre 3 y 6 para ver cómo cambia el tiempo de cómputo en cada método.

(En el Ejercicio 10 de la Unidad 2 se explica cómo usar el módulo `time`.)

16 Seamos originales

Utilizando el módulo `time`, compare el tiempo que tardan estos dos métodos para remover duplicados de una lista:

- Convirtiendo la lista a `dict` y luego de nuevo a `list`.
- Convirtiendo la lista a `set` y luego de nuevo a `list`.

¿Cuál es más rápido? ¿Qué complejidad parece tener cada uno?

Ayuda

El primer método implica crear un diccionario a partir de una lista.

Esto debe hacerse de forma tal que cada elemento en la lista pase a ser una clave, y a cada ella se le asigna un valor arbitrario (por ejemplo, `None`). De este modo, Python se encargará de que no haya claves duplicadas.

Por último, se deberá tomar el conjunto de claves y convertirlo a una lista. Esto devolverá una lista similar a la original, pero sin duplicados.

17 Altas dimensiones

Implemente un algoritmo para multiplicar dos matrices de dimensión `N x N`.

¿Qué complejidad tiene?

18 Mediana complejidad

Se tiene una lista desordenada con `2N+1` números distintos (es decir, la lista es de tamaño impar).

Implemente un algoritmo *naive* para hallar la mediana. ¿Qué complejidad tiene?

Luego, utilice un algoritmo de ordenamiento que permita resolver el problema de forma más eficiente.

> Ayuda
>
> Para una lista de tamaño `2N+1`, la mediana es el valor tal que `N` elementos son menores (o iguales) a él.

19 *Bubble sort* eficiente

La implementación de *bubble sort* en el apunte de teoría realiza más comparaciones de las necesarias. Recorre la lista de punta a punta en todas las pasadas, cuando podrían obviarse los elementos ya ubicados al final.

Implemente una versión de *bubble sort* que recorra solo la parte desordenada de la lista, sin considerar los valores ya ordenados.

20 Piedra *sort*

El algoritmo *bubble sort* recibe su nombre por la analogía con las burbujas que suben a la superficie, que en nuestro caso son los elementos de mayor valor que se ubican al final de la secuencia.

Implemente una versión "inversa" a *bubble sort* que, en vez de "subir" los elementos con mayor valor a la superficie, "hunda" los de menor valor hacia el fondo como una piedra.

21 *Selection sort* basado en máximos

La versión de *selection sort* que se discute en el apunte de teoría se basa en buscar el mínimo en la parte desordenada de la lista y trasladarlo a su ubicación final.

Implemente una versión que esté basada en la búsqueda y traslado del máximo.